INTERVIEW PREP SERIES

DSA

Interview Questions & Easy Answers

---

56 questions · 29 marked ★ most asked

Covers complexity, arrays and strings, hashing, two pointers, sliding window, linked lists, stacks and queues, trees and BST, heaps, graphs, recursion, sorting, searching and dynamic programming, plus the coding problems most often reported in fresher interview experiences (reverse a linked list, LRU cache, longest unique substring, string compression). Code is in C++, with the idea explained in plain words first.

How to use this document

- Questions go from basic to advanced inside each section. Freshers should master the ★ ones first.
- Answers are written in simple words, the way you would say them in an interview. Learn the idea, not the exact sentences.
- Questions are based on technical rounds reported by candidates on GeeksforGeeks, LeetCode Discuss, Glassdoor, Naukri and similar interview-experience pages, plus the standard question banks interviewers draw from.
- Read one section a day and say each answer out loud once. Speaking it is what makes it stick.

Complexity basics

Q1. What is time complexity and Big-O notation? ★ Most asked

Time complexity tells how the running time of an algorithm grows as the input size n grows. Big-O gives the upper bound (worst case). Common ones from fastest to slowest: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$.

Example: one loop over an array is $O(n)$; two nested loops are $O(n^2)$; binary search is $O(\log n)$.

Q2. What is space complexity?

The extra memory an algorithm uses as input grows. Creating a new array of size n is O(n) space; using a few variables is O(1). Recursion uses stack space equal to its depth.

Q3. What is the difference between best, average and worst case?

For linear search: best case O(1) (first element), average O(n/2) ≈ O(n), worst O(n) (last or missing). Interviews usually ask for the worst case.

Arrays and strings

Q4. What is the difference between an array and a linked list? ★ Most asked

- Array: contiguous memory, O(1) access by index, but inserting/deleting in the middle is O(n), and the size is fixed (static arrays).
- Linked list: nodes scattered in memory connected by pointers, O(n) access, but O(1) insert/delete when you already have the node, and dynamic size.

Q5. Reverse an array (or string) in place. ★ Most asked

Use two pointers, one at each end, swap, and move them inward. O(n) time, O(1) space.

```
void reverseArr(vector<int>& a) {
  int l = 0, r = a.size() - 1;
  while (l < r) swap(a[l++], a[r--]);
}
```

Q6. Find the maximum subarray sum (Kadane's algorithm). ★ Most asked

Keep a running sum; if it goes negative, restart from 0, because a negative prefix only hurts. Track the best seen. O(n).

```
int maxSubArray(vector<int>& a) {
  int best = a[0], cur = 0;
  for (int x : a) {
    cur = max(x, cur + x);
    best = max(best, cur);
  }
  return best;
}
```

Q7. Two Sum: find two numbers that add to a target. ★ Most asked

Brute force checks all pairs, $O(n^2)$. Better: use a hash map storing value → index. For each number, check if `target - num` is already in the map. O(n) time.

```
vector<int> twoSum(vector<int>& a, int t) {
  unordered_map<int,int> seen;
```

```
  for (int i = 0; i < a.size(); i++) {
    if (seen.count(t - a[i])) return {seen[t - a[i]], i};
    seen[a[i]] = i;
  }
  return {};
}
```

Q8. Check if a string is a palindrome. ★ Most asked

Compare characters from both ends moving inward; if any pair differs, it's not a palindrome. O(n) time, O(1) space.

Q9. Check if two strings are anagrams. ★ Most asked

Count each character's frequency in the first string and subtract for the second. If all counts are 0, they're anagrams. O(n) time. Alternatively sort both and compare (O(n log n)).

Q10. String compression: "aaabbcdeeaa" → "a3b2c1d1e2a2". ★ Most asked

Walk through the string, count how many times the current character repeats in a row, then append the character and the count.

```
string compress(const string& s) {
  string res;
  for (int i = 0; i < s.size(); ) {
    int j = i;
    while (j < s.size() && s[j] == s[i]) j++;
    res += s[i] + to_string(j - i);
    i = j;
  }
  return res;
}
```

Interview tip: This exact problem was reported in a Bititude Technologies fresher interview on Glassdoor.

Q11. Move all zeros to the end while keeping the order of other elements.

Keep a pointer `pos` for where the next non-zero should go. Copy each non-zero to `a[pos++]`, then fill the rest with zeros. O(n).

Q12. Find the missing number in an array of 1 to n.

Expected sum = n(n+1)/2. Missing number = expected sum − actual sum. O(n) time, O(1) space. (XOR also works and avoids overflow.)

Q13. Find the first non-repeating character in a string.

Count frequencies in one pass (array of 26 or a map), then scan again and return the first character with count 1. O(n).

Hashing, two pointers and sliding window

Q14. What is hashing? What is a collision? ★ Most asked

Hashing maps a key to an index using a hash function, giving O(1) average insert and lookup (unordered_map, HashMap, dict). A **collision** is when two keys map to the same index. It's handled by **chaining** (a list at each slot) or **open addressing** (probing for the next free slot).

Q15. What is the difference between `map` and `unordered_map` in C++?

`map` is a balanced BST (red-black tree): keys are sorted, operations are O(log n). `unordered_map` is a hash table: no order, O(1) average but O(n) worst case.

Q16. What is the two-pointer technique? ★ Most asked

Using two indexes that move through the data, often from both ends of a sorted array or at different speeds. It turns many $O(n^2)$ solutions into O(n). Examples: pair with given sum in a sorted array, remove duplicates, container with most water, palindrome check.

Q17. What is the sliding window technique? ★ Most asked

A window (a range `[left, right]`) moves across an array or string. You expand `right` and shrink `left` when a condition breaks, reusing work instead of recomputing. Used for subarrays/substrings: max sum of size k, longest substring without repeating characters, minimum window substring.

Q18. Longest substring without repeating characters. ★ Most asked

Use a sliding window and store each character's last index. When a repeat appears inside the window, move `left` just past its previous position. O(n).

```
int lengthOfLongestSubstring(string s) {
  vector<int> last(256, -1);
  int best = 0, left = 0;
  for (int r = 0; r < s.size(); r++) {
    if (last[s[r]] >= left) left = last[s[r]] + 1;
    last[s[r]] = r;
    best = max(best, r - left + 1);
  }
  return best;
}
```

Interview tip: Reported in a UKG SDE interview on LeetCode Discuss; they also asked to print the substring (store `left` when `best` updates).

Q19. Maximum sum of a subarray of size k.

Sum the first k elements, then slide: add the next element and subtract the one leaving the window. Track the max. O(n).

Linked lists

Q20. Reverse a linked list. ★ Most asked

Walk through the list with three pointers: `prev`, `curr`, `next`. Point each node back to `prev`. O(n) time, O(1) space.

```
ListNode* reverseList(ListNode* head) {
  ListNode *prev = nullptr, *curr = head;
  while (curr) {
    ListNode* nxt = curr->next;
    curr->next = prev;
    prev = curr;
    curr = nxt;
  }
  return prev;
}
```

Interview tip: The most reported linked-list question across Capgemini, TCS and service-company interviews.

Q21. Detect a cycle in a linked list. ★ Most asked

Floyd's algorithm (tortoise and hare): `slow` moves 1 step, `fast` moves 2 steps. If they ever meet, there is a cycle; if `fast` reaches null, there isn't. O(n) time, O(1) space.

Q22. Find the middle of a linked list. ★ Most asked

Use slow and fast pointers. When `fast` reaches the end, `slow` is at the middle. O(n).

Q23. Merge two sorted linked lists.

Use a dummy node. Compare the heads of both lists, attach the smaller one, and move forward. Attach the remaining list at the end. O(n + m).

Q24. Remove the Nth node from the end.

Move a `fast` pointer N steps ahead, then move `fast` and `slow` together until `fast` reaches the end. `slow` is now just before the node to remove. One pass, O(n).

Q25. Insert a node at the beginning, end and a position.

- Beginning: new node's next = head; head = new node. O(1).
- End: walk to the last node and set its next to the new node. O(n) (O(1) with a tail pointer).
- Position k: walk to node k−1, set newNode->next = temp->next, then temp->next = newNode.

Stacks and queues

Q26. What is the difference between a stack and a queue? ★ Most asked

A stack is **LIFO** (last in, first out), like a pile of plates: `push`, `pop`, `top`. A queue is **FIFO** (first in, first out), like a ticket line: `enqueue`, `dequeue`, `front`. Both have O(1) operations.

Q27. Check for balanced parentheses. ★ Most asked

Push every opening bracket on a stack. For a closing bracket, the stack top must be the matching opener; pop it. At the end, the stack must be empty. O(n).

```
bool isValid(string s) {
  stack<char> st;
  for (char c : s) {
    if (c == '(' || c == '{' || c == '[') st.push(c);
    else {
      if (st.empty()) return false;
      char t = st.top(); st.pop();
      if ((c == ')' && t != '(') || (c == '}' && t != '{') || (c == ']' && t != '[')) return false;
    }
  }
  return st.empty();
}
```

Q28. Implement a queue using two stacks. ★ Most asked

Use `in` for push. For pop/peek, if `out` is empty, move all elements from `in` to `out` (this reverses the order), then pop from `out`. Each element moves at most once, so it's O(1) amortized.

Interview tip: Reported in an embedUR systems interview on Glassdoor.

Q29. What are infix, prefix and postfix expressions?

Infix: `A + B` (operator between). Prefix (Polish): `+ A B`. Postfix (Reverse Polish): `A B +`. Postfix is easy for computers to evaluate with a stack: push operands; on an operator, pop two, compute, and push the result.

Q30. What is the next greater element problem?

For each element, find the first larger element to its right. Use a **monotonic stack**: go from right to left, pop smaller elements, and the stack top is the answer. O(n).

Q31. What is a deque and a priority queue?

A deque (double-ended queue) allows insert/delete at both ends in O(1). A priority queue always gives the highest (or lowest) priority element first and is implemented with a heap.

Trees and BST

Q32. What is a binary tree? What are the traversals? ★ Most asked

A tree where each node has at most two children (left and right). Traversals:

- **Inorder** (Left, Root, Right)
- **Preorder** (Root, Left, Right)
- **Postorder** (Left, Right, Root)
- **Level order** (BFS, level by level using a queue)

```
void inorder(TreeNode* root) {
  if (!root) return;
  inorder(root->left);
  cout << root->val << " ";
  inorder(root->right);
}
```

Q33. Find the height (maximum depth) of a binary tree. ★ Most asked

Height = 1 + max(height of left, height of right), with an empty tree = 0. O(n).

```
int maxDepth(TreeNode* r) {
  if (!r) return 0;
  return 1 + max(maxDepth(r->left), maxDepth(r->right));
}
```

Q34. What is a Binary Search Tree (BST)? ★ Most asked

A binary tree where, for every node, all values in the left subtree are smaller and all values in the right subtree are larger. Search, insert and delete take O(log n) on a balanced tree but O(n) if it becomes skewed (like a line). The inorder traversal of a BST is sorted.

Q35. How do you check if a tree is a valid BST?

Pass a valid range (min, max) down the tree: the left child must be < node, the right child must be > node, and every node must stay within its range. Or check that the inorder traversal is strictly increasing.

Q36. What is the Lowest Common Ancestor (LCA)?

The deepest node that has both given nodes as descendants. In a BST: if both values are smaller, go left; if both are larger, go right; otherwise the current node is the LCA. In a normal binary tree, recurse left and right; if both return non-null, the current node is the LCA.

Q37. What is a balanced tree? What is an AVL tree?

A balanced tree keeps its height close to log n. An **AVL tree** is a self-balancing BST where the heights of the left and right subtrees of any node differ by at most 1. If inserting breaks this, it fixes it with rotations (LL, RR, LR, RL).

Q38. Level order traversal (BFS) of a tree.

Use a queue: push the root; while the queue isn't empty, pop a node, process it, and push its children. To group by level, process `queue.size()` nodes at a time.

Heaps and graphs

Q39. What is a heap? ★ Most asked

A complete binary tree stored in an array. In a **min-heap** every parent is smaller than its children (smallest at the top); in a **max-heap** every parent is larger. Insert and remove-top are O(log n); peek is O(1). Used for priority queues, top-K problems and heap sort.

Q40. Find the Kth largest element in an array.

Keep a min-heap of size k. Push each element; if the size exceeds k, pop the smallest. At the end, the top is the Kth largest. O(n log k).

Q41. What is a graph? How do you represent it? ★ Most asked

A graph is a set of nodes (vertices) connected by edges. It can be directed or undirected, weighted or unweighted. Representations:

- **Adjacency matrix**: a V×V grid; O(1) edge check but $O(V^2)$ space.
- **Adjacency list**: each node stores a list of its neighbours; O(V + E) space, which is better for sparse graphs.

Q42. What is the difference between BFS and DFS? ★ Most asked

- **BFS** (Breadth-First Search) explores level by level using a **queue**. It finds the shortest path in unweighted graphs.
- **DFS** (Depth-First Search) goes as deep as possible before backtracking, using a **stack** or recursion. Used for cycle detection, topological sort and connected components.

Both are O(V + E).

Q43. What is Dijkstra's algorithm?

It finds the shortest path from a source to all nodes in a graph with **non-negative** weights. It uses a min-heap to always pick the closest unvisited node and then relaxes its edges. O((V + E) log V).

Q44. What is topological sorting?

A linear order of nodes in a Directed Acyclic Graph (DAG) where every edge u → v has u before v. Used for task scheduling and course prerequisites. Done with DFS or Kahn's algorithm (BFS using in-degrees).

Q45. What is a spanning tree and a minimum spanning tree?

A spanning tree connects all vertices of a graph using exactly V−1 edges with no cycles. A **minimum spanning tree** (MST) does this with the smallest total edge weight, found using **Kruskal's** (sort edges + union-find) or **Prim's** (grow from a node with a min-heap) algorithm.

Searching, sorting, recursion and DP

Q46. Explain binary search. ★ Most asked

Binary search works on a **sorted** array. Check the middle element: if it's the target, done; if the target is smaller, search the left half; else search the right half. O(log n).

```
int binarySearch(vector<int>& a, int t) {
  int lo = 0, hi = a.size() - 1;
  while (lo <= hi) {
    int mid = lo + (hi - lo) / 2;   // avoids overflow
    if (a[mid] == t) return mid;
    if (a[mid] < t) lo = mid + 1; else hi = mid - 1;
  }
  return -1;
}
```

Q47. Compare the common sorting algorithms. ★ Most asked

- **Bubble, Selection, Insertion**: O(n^2); simple. Insertion sort is fast on nearly sorted data.
- **Merge sort**: O(n log n) always, stable, needs O(n) extra space. Divide, sort halves, merge.
- **Quick sort**: O(n log n) average, O(n^2) worst (bad pivot), in-place, usually fastest in practice.
- **Heap sort**: O(n log n), in-place, not stable.
- **Counting/Radix sort**: O(n + k) for integers in a small range.

Q48. What is a stable sort?

A sort that keeps equal elements in their original relative order. Merge sort and insertion sort are stable; quick sort and heap sort are not.

Q49. What is recursion? What is the base case? ★ Most asked

Recursion is when a function calls itself to solve smaller versions of the same problem. The **base case** is the condition where it stops; without it, you get infinite calls and a stack overflow.

```
int factorial(int n) {
  if (n <= 1) return 1;          // base case
  return n * factorial(n - 1);   // recursive case
}
```

Q50. What is backtracking?

Trying choices one by one, and undoing (backtracking) a choice when it leads to a dead end. Used for subsets, permutations, N-Queens and Sudoku. Pattern: choose → explore (recurse) → un-choose.

Q51. What is dynamic programming (DP)? ★ Most asked

DP solves problems by breaking them into overlapping subproblems and **storing** the answers so each is solved only once. Two approaches:

- **Memoization** (top-down): recursion + cache.
- **Tabulation** (bottom-up): fill a table iteratively.

Example, Fibonacci: plain recursion is O(2^n), DP makes it O(n).

```
int fib(int n) {
  if (n < 2) return n;
  int a = 0, b = 1;
  for (int i = 2; i <= n; i++) { int c = a + b; a = b; b = c; }
  return b;
}
```

Q52. Climbing stairs: how many ways to reach step n taking 1 or 2 steps?

ways(n) = ways(n−1) + ways(n−2), the same as Fibonacci. Solve with DP in O(n) time and O(1) space.

Q53. What is the difference between greedy and dynamic programming?

Greedy picks the best option at each step and never looks back (fast, but only correct for some problems, like activity selection or coin change with standard coins). DP considers all options for subproblems and is always correct when the problem has optimal substructure (e.g. 0/1 knapsack, LCS).

Q54. Design an LRU (Least Recently Used) cache. ★ Most asked

Use a **hash map + doubly linked list**. The list keeps items in order of use (most recent at the front). The map gives O(1) access to any node.

- `get(key)`: if found, move the node to the front and return the value.
- `put(key, val)`: update or insert at the front; if over capacity, remove the node at the back (least recently used) and erase it from the map.

Both operations are O(1). In C++ you can use `list<pair<int,int>>` + `unordered_map<int, list<...>::iterator>`.

Interview tip: Reported in a UKG SDE round 2 on LeetCode Discuss. Explain the idea before writing code.

Q55. What is a Trie?

A tree for storing strings where each node is a character and paths from the root spell words. Insert and search take O(length of word). Used for autocomplete, spell checking and prefix search.

Q56. What is the union-find (disjoint set) data structure?

It keeps track of which elements are in the same group, with two operations: `find` (which group?) and `union` (merge two groups). With path compression and union by rank, both are nearly O(1). Used in Kruskal's MST and for detecting cycles in undirected graphs.